

Face Follow (No-PTZ)

1. Experiment Objective

Use OpenCV Haar Cascade to detect faces, with body rotation tracking + distance following keeping the face centered in the frame and maintaining a constant distance.

2. Experiment Principle

1. Key Terms

Term	Description
Visual Follow	Adds distance control (linear.x) on top of tracking, making the car rotate and move forward/backward simultaneously
Distance Follow	Estimates distance from the face box width, with PD control keeping the car at the desired distance
Haar Cascade	OpenCV frontal face cascade classifier for fast face region detection
PD Control	Angle PD (angular.z) + distance PD (linear.x) with independent gains
Deadband	Deadbands in both the horizontal and distance dimensions to avoid frequent actions from small deviations
QoS	best_effort (image input + cmd_vel downlink), depth=1

2. Technical Architecture

Shares the same node `face_tracker_node`, with `enable_distance_follow:=true`:

```
/camera/image_raw (BEST_EFFORT)
└→ face_tracker_node (enable_distance_follow:=true)
    ├── Grayscale → Haar Cascade → largest face → center + width
    └── Body control (10 Hz): pixel error → PD angular velocity + face width
        error → PD linear velocity → /cmd_vel
```

3. Core Topics Quick Reference

Firmware Downlink

Topic	Message Type	QoS	Field	Range	Direction
/cmd_vel	geometry_msgs/Twist	best_effort + keep_last(1)	linear.x	-0.16~0.16 m/s	+= forward (approach), -= backward (move away)
			angular.z	-1.00~1.00 rad/s	+= left turn, -= right turn

3. Experiment Steps

1. Hardware Preparation

Hardware	Requirement
PTZ Version	✗
No-PTZ Version	☑
Required Peripherals	ESP32 camera (mounted on bracket)

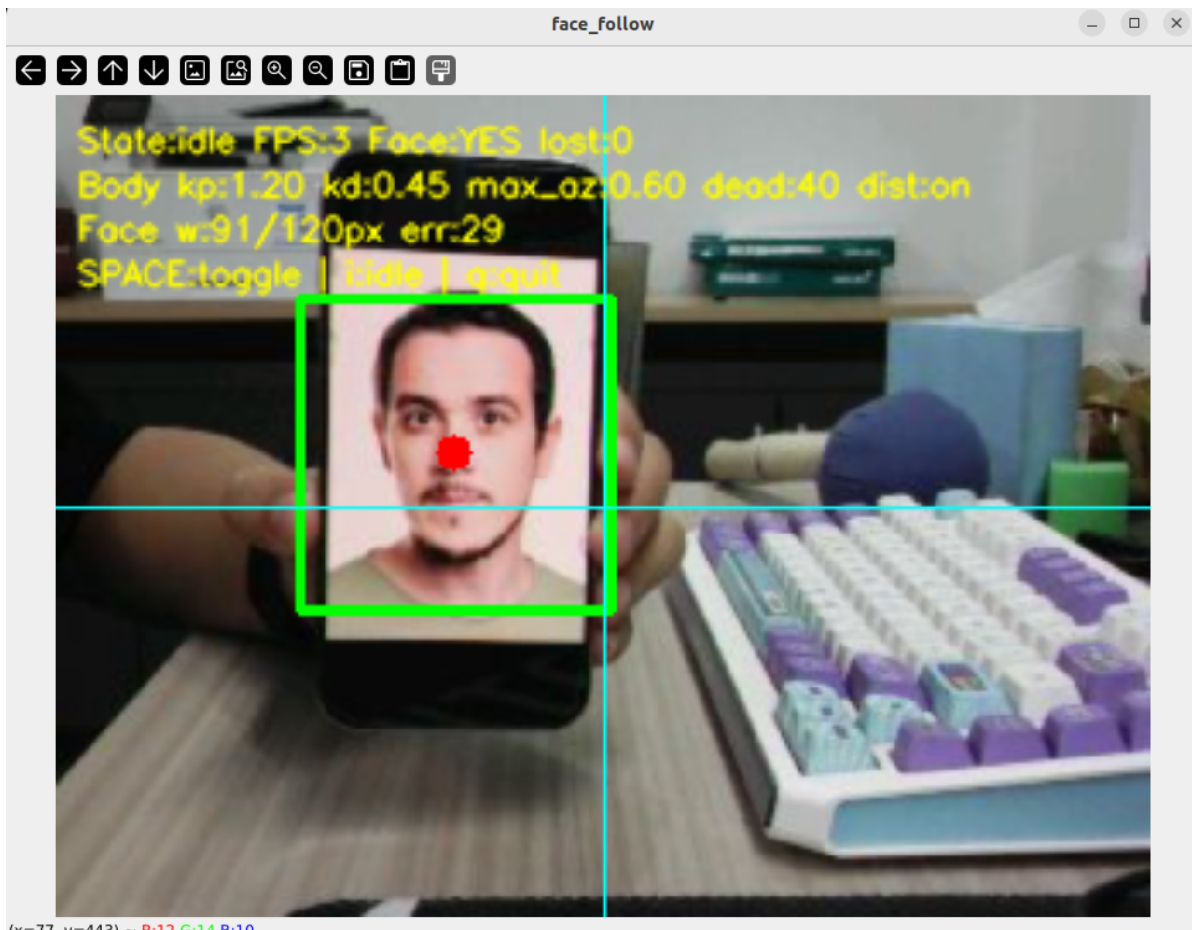
2. Start Agent + Camera + Joint Model (Terminal 1)

```
ros2 launch microros_v2_bringup car_base.launch.py
```

```
yahboom@yahboom: $ ros2 launch microros_v2_bringup car_base.launch.py
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2026-07-28-15-04-46-269642-yahboom-579370
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [micro_ros_agent-1]: process started with pid [579411]
[INFO] [camera_driver-2]: process started with pid [579413]
[INFO] [robot_state_publisher-3]: process started with pid [579415]
[INFO] [joint_state_publisher-4]: process started with pid [579417]
[INFO] [ekf_node-5]: process started with pid [579419]
[micro_ros_agent-1] [1785222287.404236] info      | UDPv4AgentLinux.cpp | init                | running...          | port: 8090
[robot_state_publisher-3] [INFO] [1785222288.847816851] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-3] [INFO] [1785222288.848013042] [robot_state_publisher]: got segment base_link
[robot_state_publisher-3] [INFO] [1785222288.848028942] [robot_state_publisher]: got segment bwheel_Link
[robot_state_publisher-3] [INFO] [1785222288.848037122] [robot_state_publisher]: got segment cam1_Link
[robot_state_publisher-3] [INFO] [1785222288.848043874] [robot_state_publisher]: got segment cam2_Link
[robot_state_publisher-3] [INFO] [1785222288.848050538] [robot_state_publisher]: got segment cam3_Link
[robot_state_publisher-3] [INFO] [1785222288.848057264] [robot_state_publisher]: got segment imu_frame
[robot_state_publisher-3] [INFO] [1785222288.848063747] [robot_state_publisher]: got segment laser_frame
[robot_state_publisher-3] [INFO] [1785222288.848070240] [robot_state_publisher]: got segment lfwheel_Link
[robot_state_publisher-3] [INFO] [1785222288.848076888] [robot_state_publisher]: got segment rfwheel_Link
[joint_state_publisher-4] [INFO] [1785222289.372349515] [joint_state_publisher]: Waiting for robot_description to be published on the robot_description
[camera_driver-2] [INFO] [1785222289.716161193] [esp32_ip_camera_publisher]: camera node started, camera_url: http://192.168.12.37:81/stream
[camera_driver-2] [WARN] [1785222292.705741414] [esp32_ip_camera_publisher]: Camera not opened, trying reconnect... (next retry in 1.0s)
[camera_driver-2] [INFO] [1785222294.072701944] [esp32_ip_camera_publisher]: IP camera stream opened successfully
```

3. Start Face Follow (Terminal 2)

```
ros2 launch microros_v2_no_ptz_visual_control_pkg face_follow.launch.py
```



5. Operation Instructions

Key	Function
Space	Toggle follow/standby
i / I	Switch to standby (IDLE)
q / ESC	Exit

Distance Follow Behavior: Face approaches (width increases) → move backward; face moves away (width decreases) → move forward; width within tolerance (± 12 px) → stay still.

4. Experiment Result

- Face approaches → the car moves backward to keep the distance
- Face moves away → the car moves forward to follow
- Face width within the desired range (120 px) → the car stays still
- Face lost → the car automatically stops once the lost-target count exceeds the threshold

5. Code Explanation

Source code paths:

- Launch:
`~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/launch/face_follow.launch.py`
- Node source:
`~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz`

1. Core Node Explanation

`face_follow.launch.py` launches `face_tracker_node`, which adds one more dimension of PD control over the tracking mode — distance PD. The node uses Haar Cascade to detect face regions and selects the face with the largest area as the target. Distance following is based on the face box width (rather than radius): width > desired → too close → move backward; width < desired → too far → move forward.

2. Key Parameters

Parameter definition files:

- Launch:
`~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/launch/face_follow.launch.py`
- Node:
`~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz_visual_control_pkg/face_tracker_node.py`

Parameter	Type	Default	Description
<code>face_min_width</code> / <code>face_min_height</code>	int	80 px	Minimum face size
<code>deadband_x_pixel</code>	float	20.0 px	Horizontal deadband
<code>angular_kp</code> / <code>angular_kd</code>	float	1.2 / 0.65	Angle PD gains
<code>max_angular_z</code>	float	1.0 rad/s	Maximum turning speed
<code>enable_distance_follow</code>	bool	true	Enable distance following
<code>target_face_width</code>	float	120.0 px	Desired face width
<code>distance_tolerance_fw</code>	float	12.0 px	Distance tolerance deadband
<code>distance_kp</code> / <code>distance_kd</code>	float	0.012 / 0.003	Distance PD gains
<code>max_linear_x</code>	float	0.16 m/s	Maximum forward/backward speed

3. Node Description

Node	Package	Description
<code>face_tracker_node</code>	<code>microros_v2_no_ptz_visual_control_pkg</code>	Face tracking + following, Haar Cascade + dual PD

6. Common Issues

Issue	Cause	Solution
Profile faces not tracked	Haar Cascade only detects frontal faces	Keep the face facing the camera
Distance following oscillates between near and far	Unreasonable PD parameters	Adjust <code>distance_kp</code> / <code>target_face_width</code>
Frequent switching with multiple faces	Largest-face strategy	Make sure only one face is in the frame